Dokumentacija za sistem zajemanja slik z RedPitayo

Martin Horvat

17. september 2025

---

1 FPGA

Program na FPGA je napisan v jeziku Verilog/SystemVerilog z Vivado 2020.2. Sistem je sestavljen iz sledečih blokov za:

- kontrolo parametrov `reg_ctrl_top`,
- zajem signala `acquire_top` in `integrator_top`,
- generiranje signala `generate_top` in `waveform_top` in
- prenos zajetega signala v RAM `AXI_HP_top`

Celotna blokovna shema je shranjena v datoteki `system.pdf`.

```
always @(posedge clk)
begin
  case (reg_addr[16-1:0])
    DEC_FACTOR_ADDR:        reg_rd_data <= cfg_dec;
    START_ACQ_ADDR:         reg_rd_data <= {31'b0, start_acq};
    DEST_ADDR:              reg_rd_data <= dest_addr;
    BUFF_SIZE_ADDR:         reg_rd_data <= buff_size;
    TRANSFER_RD_CNT_ADDR:   reg_rd_data <= {{(32-FIFO_DEPTH){1'b0}}, transfer_rd_cnt};
    FIFO_MIN_THRESH_ADDR:   reg_rd_data <= {28'b0, fifo_min_thresh};
    START_GEN_ADDR:         reg_rd_data <= {31'b0, start_gen};
    DECIMATION_GEN_ADDR_1:  reg_rd_data <= decimation_1;
    DECIMATION_GEN_ADDR_2:  reg_rd_data <= decimation_2;
    WAVEFORM_LEN_1_ADDR:    reg_rd_data <= waveform_len_1;
    WAVEFORM_LEN_2_ADDR:    reg_rd_data <= waveform_len_2;
    INTEGRATOR_DEC_ADDR:    reg_rd_data <= integrator_dec;
    BUFFER_N_ADDR:          reg_rd_data <= buffer_n;
    default                 reg_rd_data <= wfm_read;
  endcase
end
```

Slika 1: Koda za branje parametrov

```
always @(posedge clk)
begin
  if (rst_n == 0) begin
    cfg_dec <= 1;
  end else begin
    if ((reg_addr[16-1:0] == DEC_FACTOR_ADDR ) && (reg_wr_we == 1)) begin
      cfg_dec <= reg_wr_data;
    end
  end
end
```

Slika 2: Koda za zapisovanje parametrov

```
localparam DEC_FACTOR_ADDR       = 16'h0000;
localparam START_ACQ_ADDR        = 16'h0004;
localparam DEST_ADDR             = 16'h0008;
localparam BUFF_SIZE_ADDR        = 16'h000C;
localparam TRANSFER_RD_CNT_ADDR  = 16'h0010;
localparam FIFO_MIN_THRESH_ADDR  = 16'h0014;
localparam FIFO_DOUT_ADDR_1      = 16'h0018;
localparam FIFO_DOUT_ADDR_2      = 16'h001C;
localparam CTRL_REG_ADDR         = 16'h0020;
localparam INTEGRATOR_DEC_ADDR   = 16'h0024;
localparam BUFFER_N_ADDR         = 16'h0028;
localparam INPUT_SEL_ADDR        = 16'h002C;

localparam START_GEN_ADDR        = 16'h1000;
localparam DECIMATION_GEN_ADDR_1 = 16'h1004;
localparam DECIMATION_GEN_ADDR_2 = 16'h1008;
localparam WAVEFORM_LEN_1_ADDR   = 16'h100C;
localparam WAVEFORM_LEN_2_ADDR   = 16'h1010;

localparam SHIFT_WFM             = 16'h2000;

localparam CTRL_REG_INTR_ADDR       = 0;
localparam REQ_CTRL_REG_INTR_ADDR   = 1;
```

Slika 3: Definicija lokacij parametrov v delu RAMa

1.1 Kontrolni parametri

Blok komunicira s procesnim sistemom preko GP porta AXI protokola, zato ima vhode in izhode kot jih ta zahteva. Komunikacija poteka preko Xilinxovega bloka `reg_ctrl` (Slika 4), kjer lahko nato nastavitve preberemo (Slika 1) oziroma zapišemo (Slika 2). S parametri so definirane lokacije, kjer se v delu RAMa, glede na mapiranje AXI protokola nahajajo spremenljivke (Slika 3). Prebrane vrednosti oziroma registri, ki nosijo vrednost katera se zapiše v RAM, so nato vhodi in izhodi v blok `reg_ctrl_top`.

1.2 Zajem signala

Signal iz detektorja gre na en ADC vhod, na drugi vhod pa pride signal pozicije hitrega ogledala. Prvi signal gre nato skozi integrator, ki po trapeznem pravilu sešteje prispevke na vhodu v vsakem ciklu ure. Dobljena dva signala skupaj z vrednostjo počasnega signala prideta na vhod bloka `acquire_top`. Ta poskrbi, da se zgodi zapis vrednosti pozicij in integriranega signala na vsakih `dec_cnt` ciklov ure (Slika 5). Hkrati tudi ponastavi integrator.

1.3 Generiranje signala

Isti del spomina, ki služi nastavljanju nastavitvenih registrov v bloku `ctrl_reg`, je po nekem mejnem naslovu, definiranem z `WFM_SHIFT` v istem bloku, namenjen vzorcem arbitrarne funkcije, ki jo želimo generirati. In sicer je prvih 16384 vzorcev namenjenih prvi funkciji in preostalih 16384 drugi. Za zapisovanje iz RAMa v pomnilnik na FGPA poskrbi blok `waveform_top`. Vsebuje dva Xilinxova generatorja spomina - vsakega za en izhod (Slika 6). Vrednost, ki jo dobi preko bloka `reg_Ctrl` z informacijo o naslovu, s kjer prihaja vrednost, ki jo prav tako dobi od istega bloka, zapiše na ustrezno pozicijo ustreznega generatorja spomina (Slika 7).

Blok `generator_top` nato poskrbi za branje prave vrednosti iz spomina preko decimacije generatorja,

```
reg_ctrl U_reg_ctrl(
    .s_axi_aclk     (s_axi_reg_aclk),
    .s_axi_aresetn  (s_axi_reg_aresetn),
    .s_axi_awaddr   (s_axi_reg_awaddr),
    .s_axi_awprot   (s_axi_reg_awprot),
    .s_axi_awvalid  (s_axi_reg_awvalid),
    .s_axi_awready  (s_axi_reg_awready),
    .s_axi_wdata    (s_axi_reg_wdata),
    .s_axi_wstrb    (s_axi_reg_wstrb),
    .s_axi_wvalid   (s_axi_reg_wvalid),
    .s_axi_wready   (s_axi_reg_wready),
    .s_axi_bresp    (s_axi_reg_bresp),
    .s_axi_bvalid   (s_axi_reg_bvalid),
    .s_axi_bready   (s_axi_reg_bready),
    .s_axi_araddr   (s_axi_reg_araddr),
    .s_axi_arprot   (s_axi_reg_arprot),
    .s_axi_arvalid  (s_axi_reg_arvalid),
    .s_axi_arready  (s_axi_reg_arready),
    .s_axi_rdata    (s_axi_reg_rdata),
    .s_axi_rresp    (s_axi_reg_rresp),
    .s_axi_rvalid   (s_axi_reg_rvalid),
    .s_axi_rready   (s_axi_reg_rready),
    .bram_rst_a     (reg_rst),
    .bram_clk_a     (reg_clk),
    .bram_en_a      (reg_en),
    .bram_we_a      (reg_we),
    .bram_addr_a    (reg_addr),
    .bram_wrdata_a  (reg_wr_data),
    .bram_rddata_a  (reg_rd_data));
```

Slika 4: Xilinxov blok `reg_ctrl`

```
always @ (posedge clk) begin
    if (~rst_n || ~start_acq)
        dec_cnt <= 0;
    else if (trig)
        dec_cnt <= 1;
    else
        dec_cnt <= dec_cnt + 32'b1;
end
```

Slika 5: Koda za proženje zapisa in ponastavitve integratorja

ki jo nato pelje na DAC izhod.

1.4 Komunikacija zajetega signala

Najzapletenejši del logike se nahaja v delu, ki skrbi za efektiven prenos zajetega signala iz FIFO v RAM. Za to se namesto GP porta (General Purpose) uporabi port HP (High Performance). Blok

```
// Block Memory Generator instance
blk_mem_gen_0 #(
) blk_mem_inst_1 (
    .addra(bram_write_addr[14-2:0]),
    .dina(bram_din),
    .douta(bram_read_1),
    .wea(bram_we_1),
    .clka(clk),
    .addrb(bram_read_addr_1),
    .dinb(16'b0),
    .doutb(bram_dout_1),
    .web(1'b0),
    .clkb(clk)
);

// Block Memory Generator instance
blk_mem_gen_0 #(
) blk_mem_inst_2 (
    .addra(bram_write_addr[14-2:0]),
    .dina(bram_din),
    .douta(bram_read_2),
    .wea(bram_we_2),
    .clka(clk),
    .addrb(bram_read_addr_2),
    .dinb(16'b0),
    .doutb(bram_dout_2),
    .web(1'b0),
    .clkb(clk)
);
```

Slika 6: Dva blokovna spomina za vsakega od izhodov

```
assign bram_we_1 = ~bram_write_addr[ADDR_WIDTH-1] && bram_we;
assign bram_we_2 = ~bram_we_1 && bram_we;

assign bram_read = bram_write_addr[ADDR_WIDTH-1] ? bram_read_2 : bram_read_1;

always @(posedge clk)
begin
  if (rst_n == 0) begin
    bram_dout_valid <= 0;
  end else begin
    bram_dout_valid <= ~bram_we;
  end
end
```

Slika 7: Izbira pravega spomina za zapis vrednosti

`Acquire_HP_top` je sestavljen iz dveh podblokov `axi_s2mm`, ki poskrbi, da logika sledi protokolu AXI in `axi_control`, ki poskrbi za shranjevanje vzorcev v FIFO spomin in iniciacijo prenosa. Ko da blok `acquire_top` signal proženja, se vzorci iz integratorja in dveh pozicijskih signalov (Slika 8) shranijo v FIFO spomin (Slika 9). Ko število shranjenih vzorcev v FIFO spominu preseže mejno vrednost, se inicira prenos v RAM, v kolikor je zastavica `request_interrupt` nastavljena na 1 (Slika 10. Ko je prenos zaključen se postavi zastavica `intr` na 1, kar sproži interrupt in program na strani OS

prebere vzorce iz RAMa. Ko zaključi ponovno postavi zastavico `request_interrupt` na 1 in cikel se ponovi. Skoraj vsa ostala logika poskrbi, da prenos sledi protokolu.

```
assign data_in[15:0] = osc1_data;
assign data_in[31:16] = osc2_data;
assign data_in[63:32] = cnt_data;
```

Slika 8: Sestavljanje FIFO zapisa

```
fifo_generator_64_64 FIFO(
    .srst            (fifo_rst),
    .clk             (clk),
    .din             (data_in),
    .wr_en           (trigger),
    .dout            (data_out),
    .rd_en           (transfer),
    .data_count      (fifo_rd_cnt),
    .full            (full)
);
```

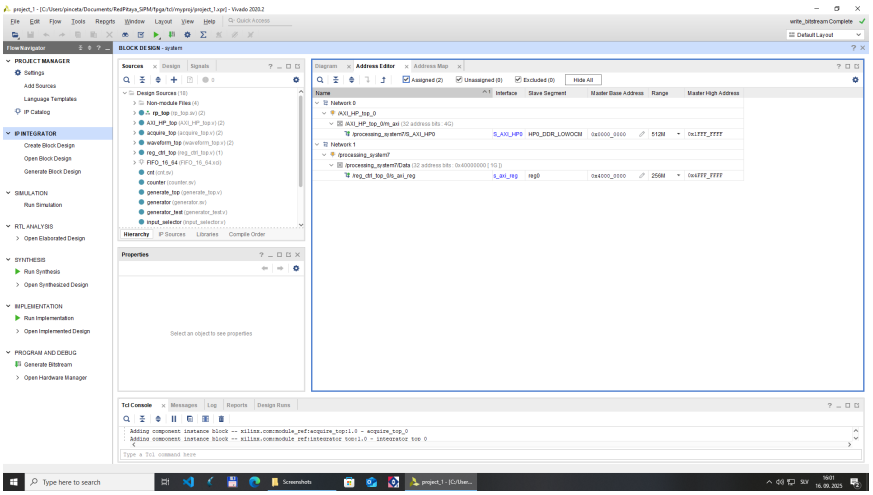
Slika 9: Zapis v fifo

```
assign data_valid = buffer_n > fifo_transfer_rd_cnt;
assign initiate_transfer = (fifo_rd_cnt > fifo_min_thresh) && (~busy) && (~request_interrupt);
assign request_interrupt = ctrl_reg[REQUEST_INTR_ADDR] && data_valid;
```

Slika 10: Zastavice za iniciacio prenosa

2 DeviceTree

Da lahko na strani OS komuniciramo z napravo, moramo definira, kjeje v RAMu se nahaja. V Vivadu sta v zavihku Address manager definirani poziciji kamor komunicirata GP in HP protokola (Slika 11). V OS dodamo definicijo naprave z devicetree overlayom, ki je prikazan na sliki 12. Relevantni deli so reg, kjer določimo pozicijo v RAM, ki se ujema s tisto nastavljeno v Vivadu in interrupts, kjer je definiran interrupt. To datoteko nato compilamo in apliciramo z ustreznima skriptama `compile_dt.sh` in `apply_overlay.sh`. To doda napravo preko generičnih UIO gonilnikov. Do naprave se lahko tedaj dostopa znotraj C programa.



Slika 11: Definicije pozicij v RAMu, kamor pišeta protokola

```
/dts-v1/;
/plugin/;
/ {
        fragment@uio {
                target = <&amba_pl>;
                __overlay__ {

                        rp_acquire: rp_acquire@50000000 {
                                #address-cells = <1>;
                                #size-cells = <1>;
                                compatible = "generic-uio";
                                reg = <0x40000000 0x1000>,<0x19000000 0x80000>;
                                        reg-names = "regset", "buffer";
                                interrupt-parent = <&intc>;
                                interrupts = <0 30 1>;
                        };

                        rp_generate: rp_generate@50100000 {
                                #address-cells = <1>;
                                #size-cells = <1>;
                                compatible = "generic-uio";
                                reg = <0x40001000 0x1000>,<0x40002000 0x10000>;
                                        reg-names = "regset", "waveform";
                                interrupt-parent = <&intc>;
                                interrupts = <0 31 1>;
                        };
                };

        };
};
```

Slika 12: DeviceTree overlay

3 C program

3.1 Identifikacija UIO naprav

V C kodi lahko najprej poiščemo vse UIO naprave v sistemu in jih shranimo na seznam. Ta seznam lahko nato preiščemo da najdemo pravo naprvao (v našem primeru `rp_acquire` in `rp_generate`). Pomožne funkcije za to so definirane v `uio_parser.h`.

3.2 Mapiranje registrov in bufferja

Ko najdemo pravo UIO napravo, lahko pozicije v spominu mapiramo na strukture (Slika 13 in 14). Strukture in funkcije za manipulacijo naprav so definirane v `generator.h` in `oscilloscope.h`. Spremenljivke znotraj strukture in njihovi relativni naslovi sovpadajo s tistimi definiranimi na sliki 3. Enako velja za waveform, kjer je spomin razdeljen na dva dela, vsakega za en izhod (Slika 14). Ko torej spremenimo katerokoli od spremenljivk znotraj mapiranih struktur, oz. jo preberemo, bo logika v FPGA poskrbela, da se sprememba zapiše oz. prebere.

```
Gen *create_gen(Uio uio) {
    char *path = malloc(sizeof(char[100]));
    snprintf(path, 100, "%s%s", "/dev/", uio.name);

    printf("%s\n", path);

    int fd = open(path, O_RDWR);

    Reg_map_gen *reg = memory_map(fd, uio.maps->uio_map.size, 0);
    Waveform_container *waveform = memory_map(fd, uio.maps->next->uio_map.size, 1);

    // Waveform_container *waveform = memory_map(fd, uio.maps->uio_map.size, 0);

    Gen *gen = malloc(sizeof(Gen));

    gen -> reg = reg;
    gen -> waveform_container = waveform;
    gen -> fd = fd;

    return gen;
}
```

Slika 13: Funkcija za dodajanje generatorja

3.3 Prenos FIFO v RAM

Ko so doseženi pogoji za prenos na FPGA strani, se začne prenos in ko se ta konča, FPGA sproži interrupt. Da ga ujamemo, moramo zanj poslušati (Slika 15). Ko ga program zazna, prenese vse podatke iz deljenega dela RAM na drugo pozicijo in zaprosi za nov interrupt (Slika 16), saj je tedaj del RAMa prost za nove podatke. Cikel se nato ponovi.

3.4 TCP komunikacija

Ko program prenese podatke iz deljenega RAM dela, jih takoj zapiše v buffer (Slika 16) in inicira TCP prenos do klienta na osebnem računalniku, kjer se sestavi slika na podlagi zgornje in spodnje meje pozicije in resolucije.

```c
typedef volatile struct {
    uint32_t start_gen;
    uint32_t dec_factor_1;
    uint32_t dec_factor_2;
    uint32_t waveform_len_1;
    uint32_t waveform_len_2;
} Reg_map_gen;

typedef volatile struct {
    int16_t waveform_1[MAX_WAVEFORM_LEN];
    int16_t waveform_2[MAX_WAVEFORM_LEN];
} Waveform_container;

typedef struct {
    Reg_map_gen *reg;
    Waveform_container *waveform_container;
    int fd;
} Gen;
```

Slika 14: Definicija struktur za generator

```c
while (wait(acq) == 0) {
    /*// clock_gettime(CLOCK_MONOTONIC, &ts_end);···

    len = acq->reg->buff_n*8;

    /*// bzero(buff, BUFF_LEN);···
    write(connfd, acq->buff, len);
    request_and_clear_interrupt(acq);
}
```

Slika 15: Poslušanje za interrupt znotraj TCP zanke

```c
void request_and_clear_interrupt(Acq *acq) {
    // printf("Interrupt requested!\n");
    set_reg(&(acq->reg->ctrl_reg), 0x00000003);
}
```

Slika 16: Brisanje interrupta in zaprošanje za novega